

2일차(2)

날짜	2026년 7월 3일
상태	완료
시간	3
요약	Amazon Bedrock & LLM 대화의 기본 구조 이해 [실습] Amazon Bedrock 기초 & Foundation Model 탐색 + 자연어처리 기초 & 프롬프트 엔지니어링 + Amazon Bedrock을 활용한 학교 FAQ 챗봇 구현

Amazon Bedrock

[attachment:71261c8e-3545-4fde-b709-ce483249db1e:Slide-04_AWS_Bedrock_v2.pdf](#)

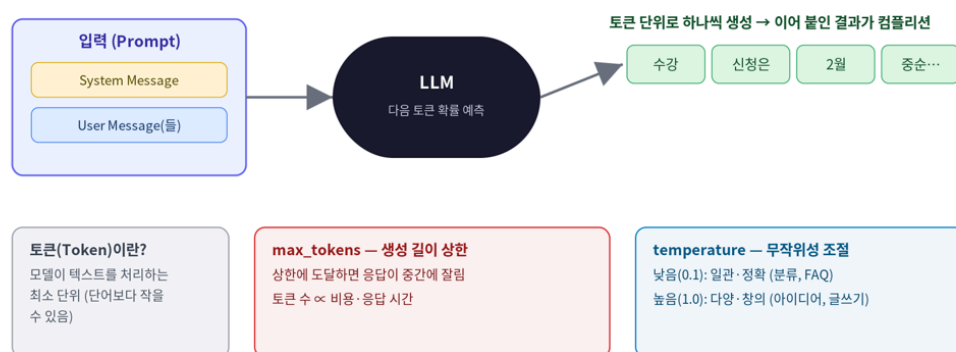
Amazon Bedrock: Foundation model을 API 호출해서 생성형 AI 애플리케이션을 구축/보강/운영하도록 제공하는 관리형 서비스

LLM 대화의 기본 구조 이해

컴플리션(Completion): 모델이 응답을 만드는 방식

응답 → 주어진 입력(프롬프트)을 이어받아 다음에 올 확률이 가장 높은 토큰을 하나씩 생성 하고 그걸 이어 붙임

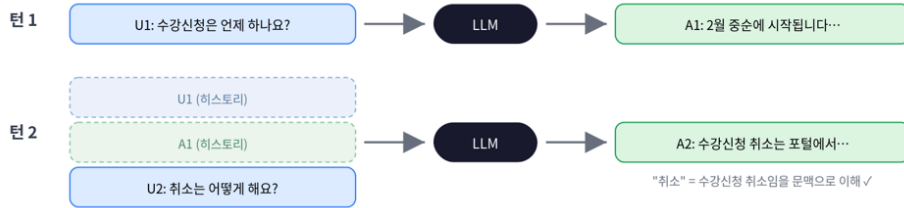
컴플리션(Completion) — 모델이 응답을 만드는 과정



멀티턴(Multi-turn): 문맥이 이어지는 대화

모델 → 상태가 없음(stateless) → 매 호출마다 대화 히스토리 전체를 다시 보냄...

멀티턴 대화 — 매 호출마다 히스토리 전체를 다시 보낸다



핵심 포인트

1. 모델은 상태가 없다(stateless) - 이전 대화를 기억하지 못하므로, 매 호출에 히스토리 전체를 messages로 전달
2. 히스토리는 user / assistant 메시지가 번갈아 쌓인 리스트 - 응답이 오면 assistant 메시지로 추가
3. 턴이 늘수록 입력 토큰이 커짐 - 비용·속도·컨텍스트 한도 문제 -> 최근 N개만 유지 (lab3의 max_history)
4. System Message는 히스토리에도 넣지 말고 호출마다 별도에 저장 (converse의 system 파라미터)
5. 새 주제로 시작하려면 히스토리를 비우면 됨 (lab3의 reset())

[실습1] Amazon Bedrock 기초 & Foundation Model 탐색

Bedrock 클라이언트 (모델 목록 조회용)

```
bedrock = boto3.client('bedrock', region_name='us-east-1')
```

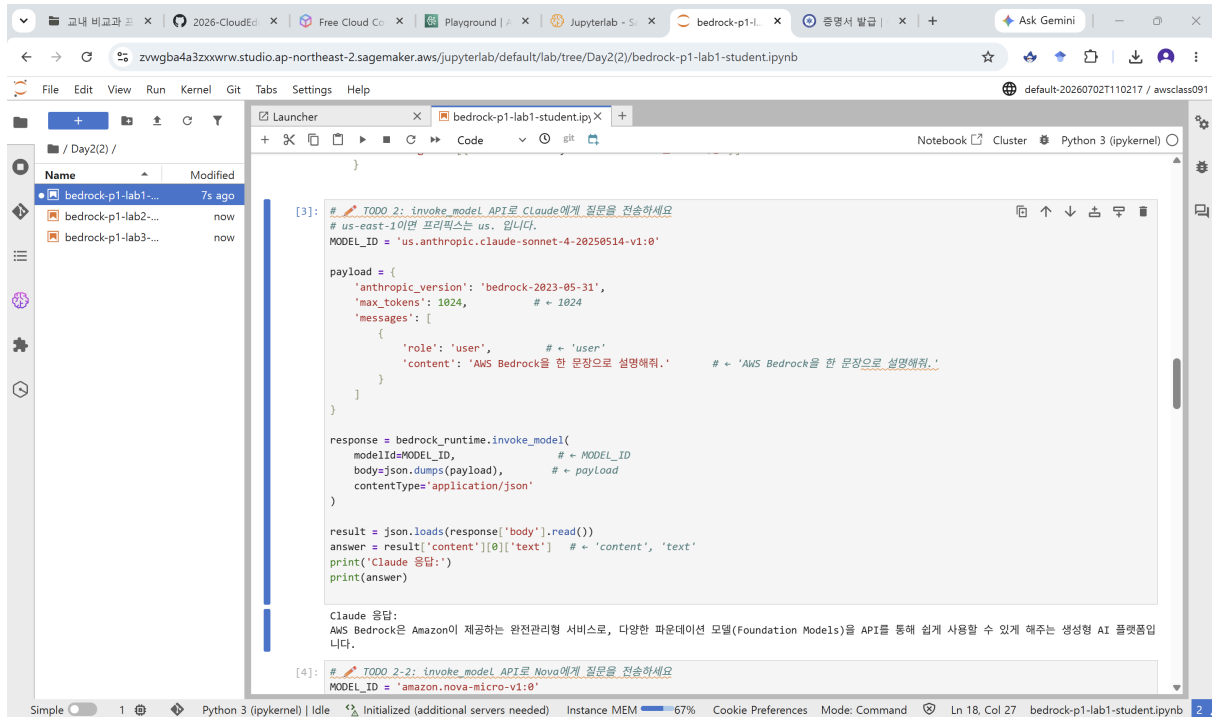
```
bedrock.list_foundation_models(byOutputModality='TEXT')
```

- 사용 가능한 모델을 조회

[illegible]

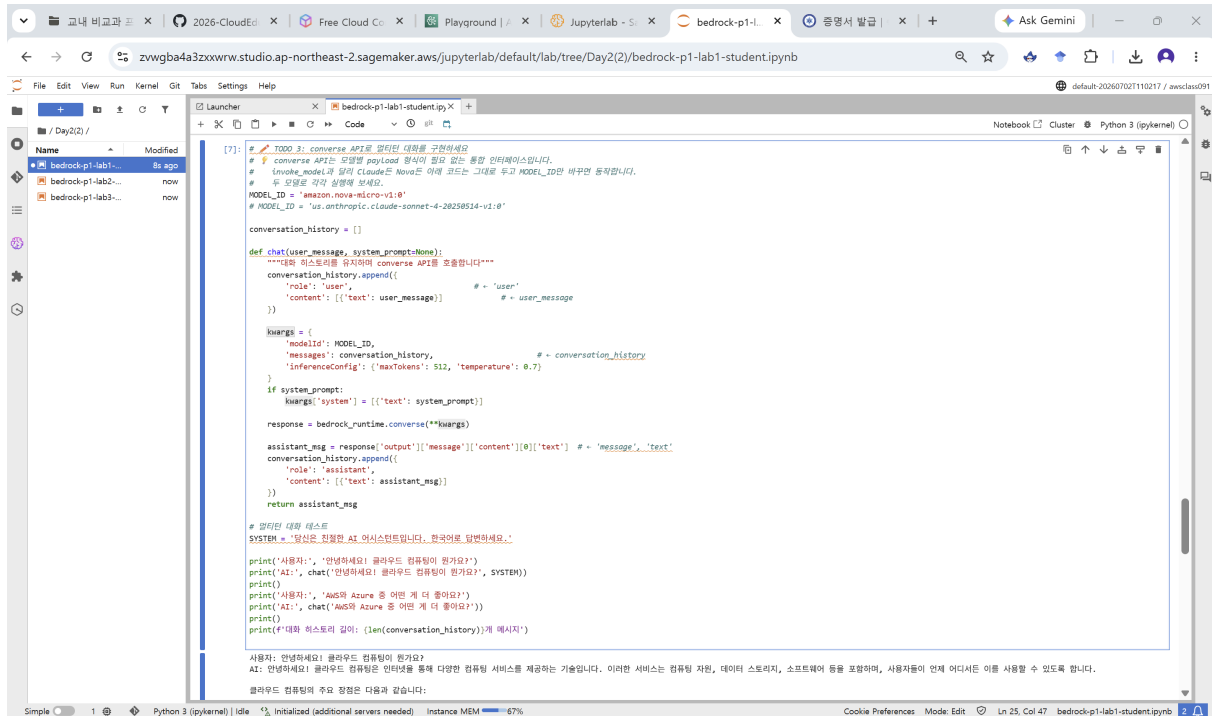
```
# Claude 3 페이로드 구조
{
  'anthropic_version': 'bedrock-2023-05-31',
  'max_tokens': 1024,
  'messages': [{ 'role': 'user', 'content': '프롬프트 내용' }]
}
```

-> Payload 형식과 불러오는 Model이 동일해야함



```
bedrock_runtime.converse(
    modelId='...',
    messages=[{'role': 'user', 'content': [{'text': '...'}]}],
    inferenceConfig={'maxTokens': 512, 'temperature': 0.7}
)
```

- 모델별 페이로드 차이를 추상화



[실습 2] 자연어처리 기초 & 프롬프트 엔지니어링

기법	언제 사용
Zero-shot	간단하고 명확한 작업
Few-shot	특정 형식/스타일이 필요할 때
Chain-of-Thought	추론·계산이 필요한 복잡한 문제
System Prompt	일관된 AI 행동 제어
Structured Output	프로그램에서 파싱할 때

```

zero_shot_template = """ """
few_shot_template = """ (예시 포함) """

# zero-shot 호출
claude(zero_shot_template.format(review=review), temperature= )

# Few-shot 호출
claude(few_shot_template.format(review=review), temperature= )

```

The screenshot shows a JupyterLab notebook with the following code:

```

이제 다음 리뷰를 분류하세요.
리뷰: {'review': ' '}
'''

# 리뷰의 주제는 별자 (모음이 성명을 덧붙여도 표가 깨지지 않도록)
def to_label(text):
    for label in ('Good', 'Bad', 'Mixed'):
        if label in text:
            return label
    return text.replace('\n', ' ')[0:20]

# 피크리온 표본 출력 - 생 출력을 주석에 피크리온 메타데이터로 붙여넣을 수 있습니다
rows = [['리뷰 | Zero-shot | Few-shot |'], ['---|---|---|']]
for review in test_reviews:
    # Zero-shot 결과
    zero_result = claude(
        zero_shot_template.format(review=review), # = review=review
        temperature=0.1 # = 0.1 (유관한 분류를 위해 낮게)
    ).strip()

    # Few-shot 결과
    few_result = claude(
        few_shot_template.format(review=review), # = review=review
        temperature=0.1 # = 0.1
    ).strip()

    rows.append([' | 리뷰 | Zero-shot | Few-shot |',
                ['---|---|---|'],
                [' | 이 강의 정말 유익했어요. 내용도 알차고 강사님도 친절하세요. | 긍정 | 긍정 |',
                 [' 과제가 너무 많고 설명이 부족해서 따라가기 힘들었습니다. | 부정 | 부정 |',
                 [' 그냥 평범한 수업이에요. 특별히 좋거나 나쁘지 않았습니다. | 중립 | 긍정 |']

markdown_table = '\n'.join(rows)
print(markdown_table) # = 이 텍스트를 복사해서 사용

# 노트북 안에서 렌더링 결과도 새로 확인
from IPython.display import Markdown, display
display(Markdown(markdown_table))

```

The output table is as follows:

리뷰	Zero-shot	Few-shot
이 강의 정말 유익했어요. 내용도 알차고 강사님도 친절하세요.	긍정	긍정
과제가 너무 많고 설명이 부족해서 따라가기 힘들었습니다.	부정	부정
그냥 평범한 수업이에요. 특별히 좋거나 나쁘지 않았습니다.	중립	긍정

`cot_prompt = f"""`

다음 문제를 단계별로 풀어주세요.

규칙:

1. 각 계산 단계를 [단계 N] 형식으로 표시하세요.
2. 각 단계에서 수식과 결과를 명시하세요.
3. 마지막에 [최종 답변]을 반드시 제시하세요.

문제: {problem}

"""

`claude(cot_prompt, temperature=)`

```
File Edit View Run Kernel Git Tabs Settings Help
bedrock-p1-lab2-student.ipynb
[3]: # TODO 2: CoT 프롬프트를 작성하여, 단계별 추론을 유도하세요
problem = """
과제가 너무 많고 일정이 무척이나 바빠서 따라가기가 힘들었습니다. | 부정 | 부정 |
그냥 답변만 부탁드립니다. | 중립 | 중립 | 긍정 |

TODO 2: Chain-of-Thought 프롬프팅
복합한 문제를 단계별로 추론하도록 Chain-of-Thought 프롬프트를 작성하세요.

[3]: # TODO 2: CoT 프롬프트를 작성하여, 단계별 추론을 유도하세요
problem = """
학교 AI 강의 수강생이 20명이다.
첫 번째 과제를 제출한 학생은 전체의 80%이다.
그 중 60%가 두 번째 과제도 제출했다.
두 번째 과제를 제출하지 않은 학생은 몇 명인가?
"""

# 일반 프롬프트 (CoT 없음)
normal_prompt = f'다음 문제를 물어주세요: {problem}'

# CoT 프롬프트 - 빈칸 완성
cot_prompt = f"""
다음 문제를 단계별로 물어주세요.

규칙:
1. 각 계산 단계를 [단계 N] 형식으로 표시하세요.
2. 각 단계에서 수식과 결과를 명시하세요.
3. 마지막에 [최종 답변]을 반드시 제시하세요.

문제: {problem}
"""

print('=== 일반 프롬프트 결과 ===')
print(encode(normal_prompt, temperature=0.1))
print()
print('=== Chain-of-Thought 결과 ===')
print(encode(cot_prompt, temperature=0.1)) # = cot_prompt

=== 일반 프롬프트 결과 ===
주어진 정보를 정리하여 단계별로 물어보겠습니다.

**주어진 정보:**
- 전체 수강생: 20명
- 첫 번째 과제 제출률: 80%
- 첫 번째 과제 제출자 중 두 번째 과제도 제출한 비율: 60%

**단계별 풀이:**
1) **첫 번째 과제를 제출한 학생 수**
   - 20명 * 80% = 20 * 0.8 = 16명

2) **두 번째 과제를 제출한 학생 수**
   - 첫 번째 과제 제출자 중 60%가 두 번째 과제도 제출
   - 16명 * 60% = 16 * 0.6 = 9.6명 → **10명** (반올림)

3) **두 번째 과제를 제출하지 않은 학생 수**
   - 전체 학생 수 - 두 번째 과제 제출 학생 수
   - 20명 - 10명 = **10명**

따라서 두 번째 과제를 제출하지 않은 학생은 **10명**입니다.

=== Chain-of-Thought 결과 ===
주어진 문제를 단계별로 해결하였습니다.

[단계 1] 첫 번째 과제를 제출한 학생 수 계산
- 전체 학생 수: 20명
- 첫 번째 과제 제출률: 80%
- 첫 번째 과제 제출 학생 수 = 20 * 0.8 = 16명

[단계 2] 두 번째 과제를 제출한 학생 수 계산
- 첫 번째 과제를 제출한 학생 중 60%가 두 번째 과제도 제출
- 두 번째 과제 제출 학생 수 = 16 * 0.6 = 9.6명
- 실제로는 9명 또는 10명이어야 하므로, 문제의 조건상 10명으로 계산

[단계 3] 두 번째 과제를 제출하지 않은 학생 수 계산
- 전체 학생 수: 20명
- 두 번째 과제를 제출한 학생 수: 16 * 0.6 = 9.6명 → 10명
- 두 번째 과제를 제출하지 않은 학생 수 = 20 - 10 = 10명

[최종 답변]
두 번째 과제를 제출하지 않은 학생은 **10명**입니다.
```

```
File Edit View Run Kernel Git Tabs Settings Help
bedrock-p1-lab2-student.ipynb
print('=== 일반 프롬프트 결과 ===')
print(encode(normal_prompt, temperature=0.1))
print()
print('=== Chain-of-Thought 결과 ===')
print(encode(cot_prompt, temperature=0.1)) # = cot_prompt

=== 일반 프롬프트 결과 ===
주어진 정보를 정리하여 단계별로 물어보겠습니다.

**주어진 정보:**
- 전체 수강생: 20명
- 첫 번째 과제 제출률: 80%
- 첫 번째 과제 제출자 중 두 번째 과제도 제출한 비율: 60%

**단계별 풀이:**
1) **첫 번째 과제를 제출한 학생 수**
   - 20명 * 80% = 20 * 0.8 = 16명

2) **두 번째 과제를 제출한 학생 수**
   - 첫 번째 과제 제출자 중 60%가 두 번째 과제도 제출
   - 16명 * 60% = 16 * 0.6 = 9.6명 → **10명** (반올림)

3) **두 번째 과제를 제출하지 않은 학생 수**
   - 전체 학생 수 - 두 번째 과제 제출 학생 수
   - 20명 - 10명 = **10명**

따라서 두 번째 과제를 제출하지 않은 학생은 **10명**입니다.

=== Chain-of-Thought 결과 ===
주어진 문제를 단계별로 해결하였습니다.

[단계 1] 첫 번째 과제를 제출한 학생 수 계산
- 전체 학생 수: 20명
- 첫 번째 과제 제출률: 80%
- 첫 번째 과제 제출 학생 수 = 20 * 0.8 = 16명

[단계 2] 두 번째 과제를 제출한 학생 수 계산
- 첫 번째 과제를 제출한 학생 중 60%가 두 번째 과제도 제출
- 두 번째 과제 제출 학생 수 = 16 * 0.6 = 9.6명
- 실제로는 9명 또는 10명이어야 하므로, 문제의 조건상 10명으로 계산

[단계 3] 두 번째 과제를 제출하지 않은 학생 수 계산
- 전체 학생 수: 20명
- 두 번째 과제를 제출한 학생 수: 16 * 0.6 = 9.6명 → 10명
- 두 번째 과제를 제출하지 않은 학생 수 = 20 - 10 = 10명

[최종 답변]
두 번째 과제를 제출하지 않은 학생은 **10명**입니다.
```

SYSTEM_JSON = """

당신은 텍스트 분석 AI입니다.

반드시 다음 JSON 형식으로만 응답하세요. JSON 외 다른 텍스트는 절대 포함하지 마세요.

```
{
  "sentiment": "긍정|부정|중립",
  "confidence": 0.0~1.0,
  "keywords": ["키워드1", "키워드2", "키워드3"],
```

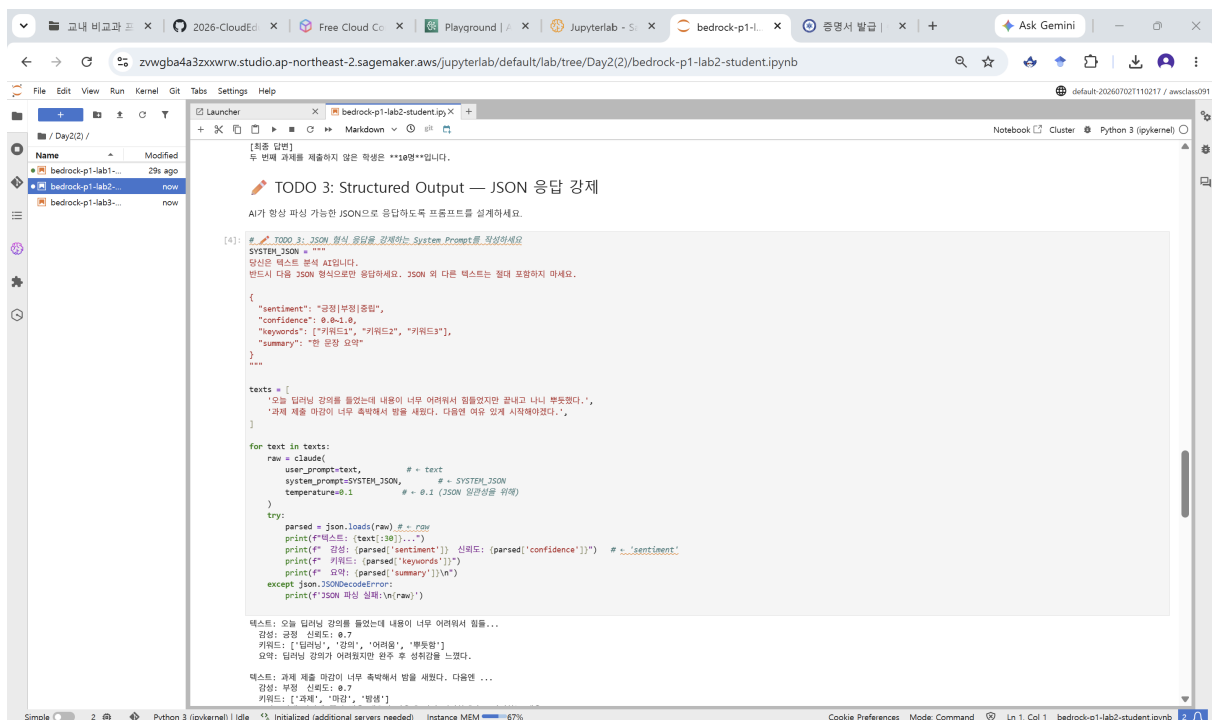
```

"summary": "한 문장 요약"
}
"""

texts = [
    '오늘 딥러닝 강의를 들었는데 내용이 너무 어려워서 힘들었지만 끝내고 나니 뿌듯했다.',
    '과제 제출 마감일이 너무 촉박해서 밤을 새웠다. 다음엔 여유 있게 시작해야겠다.',
]

claude(user_prompt=text, system_prompt=SYSTEM_JSON, temperature= )

```



[실습 3] Amazon Bedrock을 활용한 학교 FAQ 챗봇 구현

```

# 키워드 기반 검색 함수 (Retrieval)
def search_faq(query, top_k=3):
    results = []
    for category in FAQ_DATA:
        # 카테고리 키워드가 쿼리에 얼마나 포함되는지로 점수 매기기
        category_score = sum(1 for kw in category['keywords'] if kw in query)

        for qa in category['qa']:
            # 항목의 단어가 쿼리에 포함되는지로 점수 매기기 (가중치 다르게)
            q_score = sum(1 for word in qa['q'].split() if word in query)
            a_score = sum(1 for word in qa['a'].split() if word in query)
            total_score = category_score * 2 + q_score * 3 + a_score

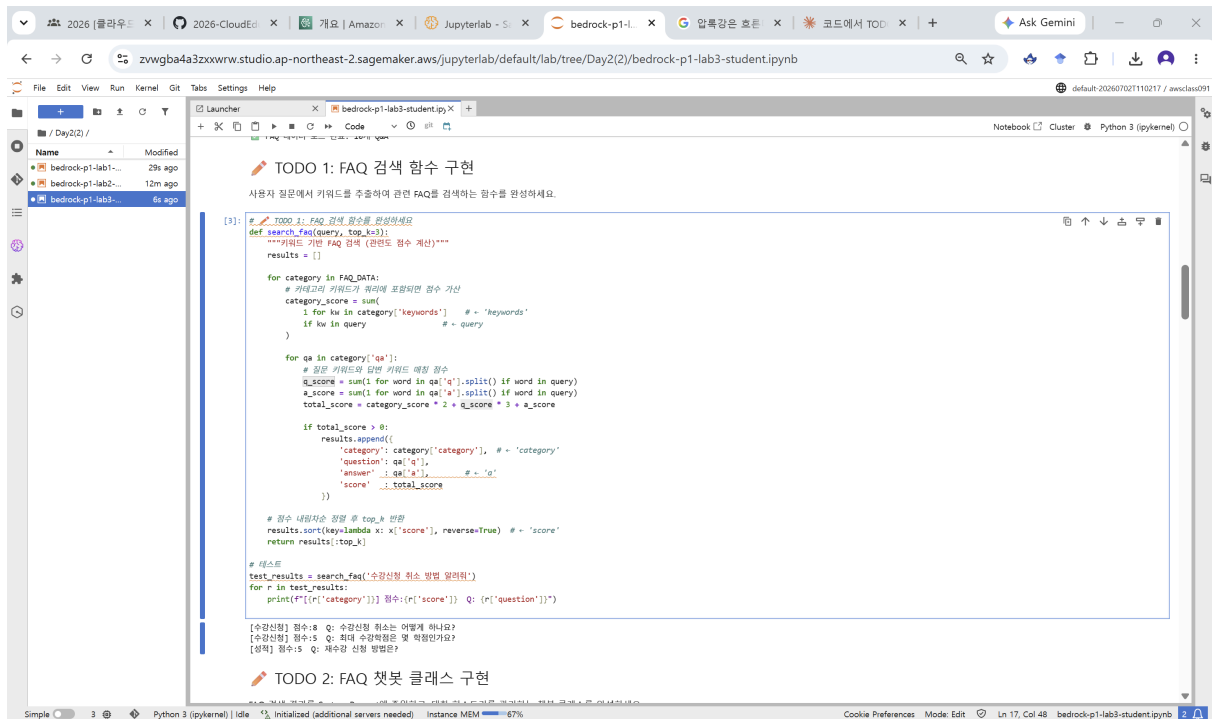
```

```

if total_score > 0:
    results.append({... , 'score': total_score})

results.sort(key=lambda x: x['score'], reverse=True) # 점수 내림차순 정렬
return results[:top_k]

```



```

# 컨텍스트 주입 챗봇 클래스 (RAG 축소판)
class Chatbot:
    def __init__(self, model_id, max_history=10):
        self.history = [] # 멀티턴 메시지 저장

    def _build_system_prompt(self, search_results):
        base = "역할/규칙 정의"
        if search_results:
            context = "\n\n[검색된 근거 정보]\n\n"
            for r in search_results:
                context += f"Q: {r['question']}\nA: {r['answer']}\n\n"
            return base + context
        return base

    def chat(self, user_message):
        results = search_faq(user_message) # 1. 검색(Retrieval)

        system_prompt = self._build_system_prompt(results) # 2. 주입(Augmentation)

```



```

        self.history.append({'role': 'user', 'content': [{'text': user_message}]}))
        if len(self.history) > self.max_history:
            # 대화 히스토리 관리(길이 제한 포함)
            self.history = self.history[-self.max_history:]

        response = self.client.converse(
            # 3. 생성(converse 호출)
            modelId=self.model_id,
            messages=self.history,
            system=[{'text': system_prompt}],
            inferenceConfig={'maxTokens': 512, 'temperature': 0.3}
        )
        answer = response['output']['message']['content'][0]['text']
        # 응답을 히스토리에 다시 저장
        self.history.append({'role': 'assistant', 'content': [{'text': answer}]}))
        return answer, results

```

```

# 멀티턴 실행
bot = FAQChatbot()
for q in questions:
    answer, refs = bot.chat(q)

```

The screenshot shows a JupyterLab interface with a notebook titled 'TODO 3: 챗봇 실행 & 멀티턴 테스트'. The notebook contains a Python script that initializes an FAQChatbot and processes a list of questions. The output shows the chatbot's responses to various queries, including information about the course schedule and fees.

```

[6]: # TODO 3: FAQChatbot을 생성하고, 멀티턴 대화를 테스트하세요
bot = FAQChatbot()

questions = [
    '수강신청은 언제 하나요?',
    '회소는 어떻게 해요?',
    '성적 장학금 기준도 알려주세요',
    '졸업하려면 학점이 몇 개 필요해요?',
]

for q in questions:
    print(f'❗ 학생: {q}')
    answer, refs = bot.chat(q)
    print(f'🗨 챗봇: {answer}')
    if refs:
        print(f'📄 참조 FAQ: {refs[0]["category"]} - {refs[0]["question"][:25]}...')

```

학생: 수강신청은 언제 하나요?
챗봇: 안녕하세요! 수강신청 일정에 대해 안내드리겠습니다.
 수강신청은 매 학기 개강 2주 전에 진행됩니다. 구체적인 일정은 다음과 같습니다:
 - 1학년: 3월 1일~3일
 - 2학년 이상: 2월 27일~28일
 미리 수강 계획을 세우고 해당 기간에 놓치지 않도록 주의해 주세요. 추가 문의사항이 있으시면 언제든지 말씀해 주세요!
 [참조 FAQ: 수강신청 - 수강신청은 언제 하나요?...]

학생: 회소는 어떻게 해요?
챗봇: 안녕하세요! 수강신청 취소에 대해 안내드리겠습니다.
 수강신청 취소는 학생포탈 → 수강관리 → 수강취소 메뉴에서 가능합니다. 개강 후 2주 이내까지 취소할 수 있으나 기간을 놓치지 않도록 주의해 주세요.
 취소 절차가 궁금하시거나 추가 문의사항이 있으시면 언제든지 말씀해 주세요!
 [참조 FAQ: 수강신청 - 수강신청 취소는 어떻게 하나요?...]

학생: 성적 장학금 기준도 알려주세요
챗봇: 안녕하세요! 성적 장학금 기준에 대해 안내드리겠습니다.
 성적 장학금을 받기 위해서는 지난 학기에 12학점 이상을 수강해야 합니다. 항목에 따른 장학금 지급 기준은 다음과 같습니다:
 - 평점 3.8 이상: 전액 장학금
 - 평점 3.5~3.79: 반액 장학금
 성적 관리를 통해 장학금 혜택을 받으시길 바랍니다!
 [참조 FAQ: 장학금 - 성적 장학금 기준은?...]

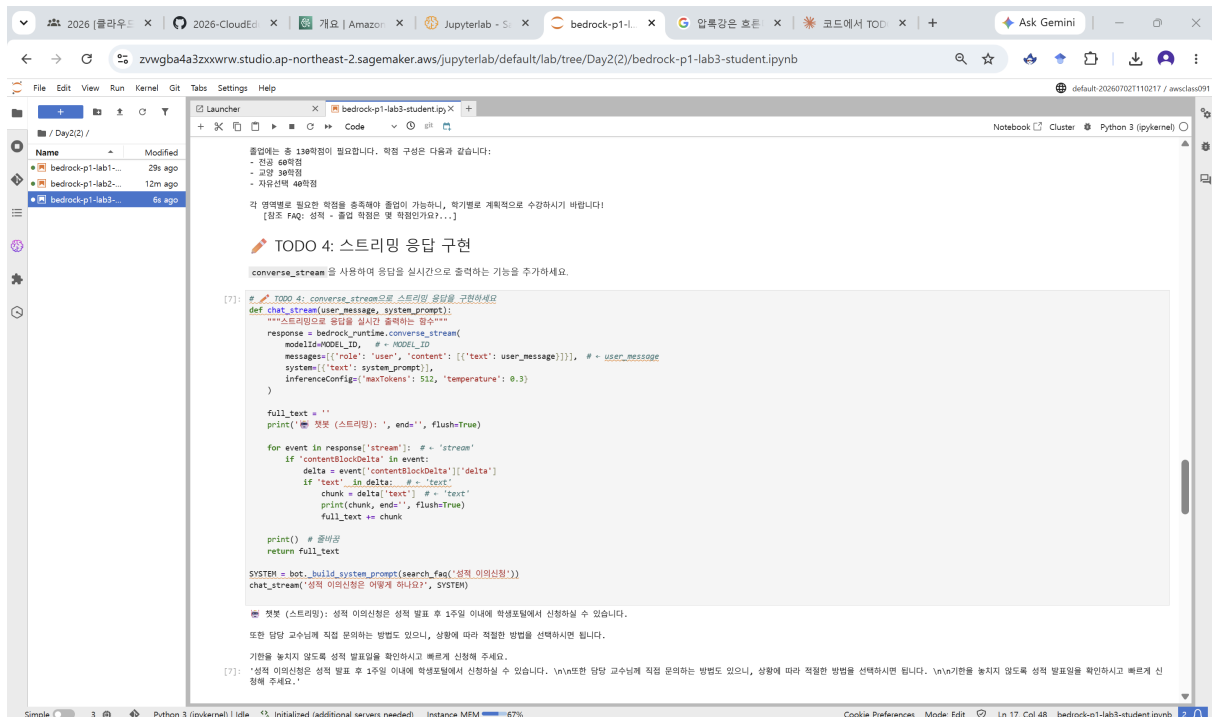
스트리밍 응답

```
response = bedrock_runtime.converse_stream(
    modelId=MODEL_ID,
    messages=[{'role': 'user', 'content': [{'text': user_message}]}],
    system=[{'text': system_prompt}],
    inferenceConfig={'maxTokens': , 'temperature': }
)
```

```
full_text = ''
for event in response['stream']:
    if 'contentBlockDelta' in event:
        delta = event['contentBlockDelta']['delta']
        if 'text' in delta:
            chunk = delta['text']
            print(chunk, end='', flush=True)
            full_text += chunk
```

```
user_message = input() # 입력받기
SYSTEM = bot._build_system_prompt(search_faq(user_message))
chat_stream(user_message, SYSTEM)
```

→ Bedrock 스트리밍 표준 처리



```
[7]: # TODO 4: converse_stream으로 스트리밍 응답을 구현하세요
def chat_stream(user_message, system_prompt):
    """스트리밍으로 응답을 실시간으로 출력하는 함수"""
    response = bedrock_runtime.converse_stream(
        modelId=MODEL_ID, # MODEL_ID
        messages=[{'role': 'user', 'content': [{'text': user_message}]}], # user_message
        system=[{'text': system_prompt}], # system_message
        inferenceConfig={'maxTokens': 512, 'temperature': 0.3}
    )

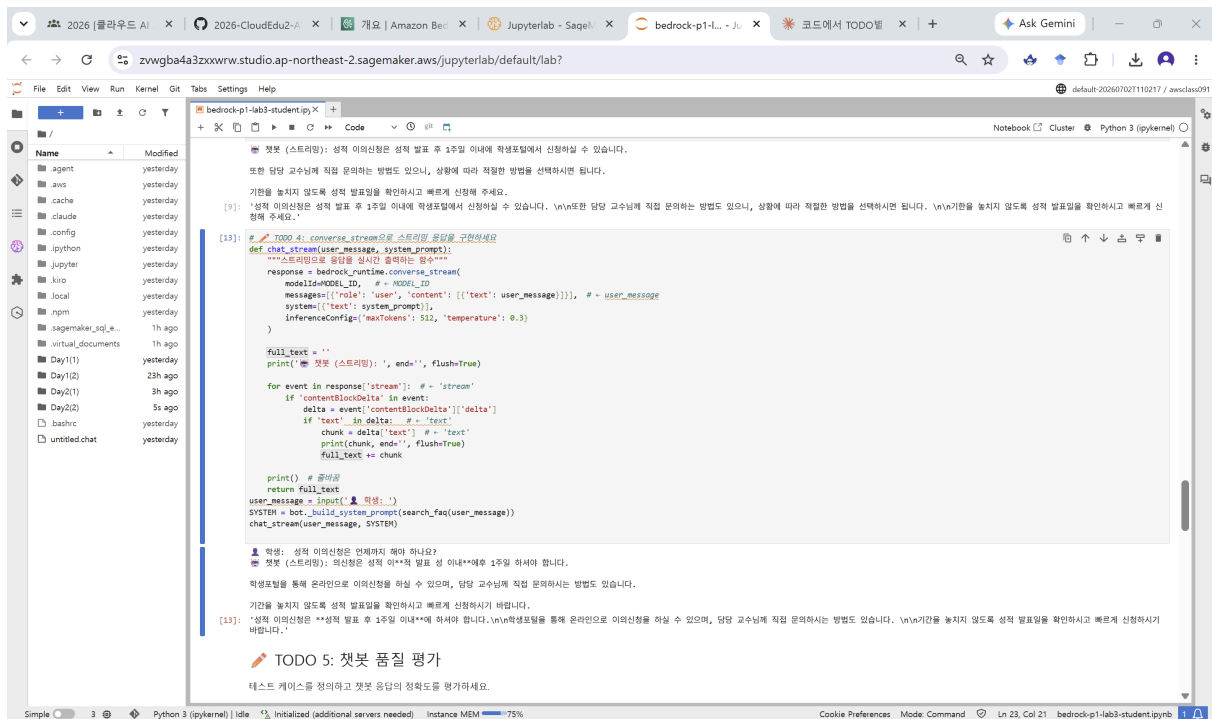
    full_text = ''
    print(f"첫 번째 스트리밍: ", end='', flush=True)

    for event in response['stream']:
        if 'contentBlockDelta' in event:
            delta = event['contentBlockDelta']['delta']
            if 'text' in delta:
                chunk = delta['text']
                print(chunk, end='', flush=True)
                full_text += chunk

    print() # 줄바꿈
    return full_text

SYSTEM = bot._build_system_prompt(search_faq('성적 의미심장'))
chat_stream('성적 의미심장은 어떻게 하나요?', SYSTEM)

# 첫 번째 스트리밍: 성적 의미심장은 성적 발표 후 1주일 이내에 학생포털에서 신청하실 수 있습니다.
또한 담당 교수님께 직접 문의하는 방법도 있으나, 상황에 따라 적절한 방법을 선택하시면 됩니다.
기한을 놓치지 않도록 성적 발표일을 확인하시고 빠르게 신청해 주세요.
[7]: '성적 의미심장은 성적 발표 후 1주일 이내에 학생포털에서 신청하실 수 있습니다. \n\n또한 담당 교수님께 직접 문의하는 방법도 있으나, 상황에 따라 적절한 방법을 선택하시면 됩니다. \n\n기한을 놓치지 않도록 성적 발표일을 확인하시고 빠르게 신청해 주세요.'
```



품질 평가 (키워드 매칭 기반)

```
def evaluate_chatbot(bot, test_cases):
```

```
    results = []
```

```
    bot.reset() # 평가 전 히스토리 초기화 → 테스트 간 독립성 보장
```

```
    for tc in test_cases:
```

```
        answer, _ = bot.chat(tc['question'])
```

```
        found = [kw for kw in tc['expected_keywords'] if kw in answer]
```

```
        passed = len(found) > 0 # 응답에 기대 키워드가 하나라도 포함되면 통과
```

```
        results.append({...})
```

```
df = pd.DataFrame(results)
```

```
pass_rate = (df['통과'] == '✅').mean() * 100 # 통과율 집계
```

```
return df
```

2026 (월)라우드 X 2026-CloudEd X 개요 | Amazon X Jupyterlab - S X bedrock-p1-l... X 압록강은 흐른 X 코트에서 TOD X Ask Gemini

zvwgba4a3zxwrrw.studio.ap-northeast-2.sagemaker.aws/jupyterlab/default/lab/Day2(2)/bedrock-p1-lab3-student.ipynb

File Edit View Run Kernel Git Tabs Settings Help

Launcher Code Python 3 (pykernel)

bedrock-p1-lab3-student.ipynb

TODO 5: 챗봇 품질 평가

테스트 케이스를 정의하고 챗봇 응답의 정확도를 평가하세요.

```
[8]: # TODO 5: 챗봇 품질 평가 함수를 완성하세요

TEST_CASES = [
    {'question': '수강신청 최대 학점만?', 'expected_keywords': ['21학점', '24학점']},
    {'question': '성적 이의신청 기간은?', 'expected_keywords': ['1주일']},
    {'question': '장학금 성적 기준 알려줘', 'expected_keywords': ['3.8', '3.5']},
    {'question': '오늘 점심 메뉴 추천해줘', 'expected_keywords': ['학생자', '문의'], # 범위 외 질문
}

def evaluate_chatbot(bot, test_cases):
    """테스트 케이스 기반 챗봇 평가"""
    results = []
    bot.reset()

    for tc in test_cases:
        answer, _ = bot.chat(tc['question']) # = 'question'

        # 기대 키워드 포함 여부 확인
        found = [kw for kw in tc['expected_keywords'] if kw in answer] # = answer
        passed = len(found) > 0 # = 0

        results.append({
            '질문': tc['question'],
            '결과': 'O' if passed else 'X',
            '대칭키워드': str(found),
            '응답일부': answer[:50] + '...'
        })

    import pandas as pd
    df = pd.DataFrame(results)
    pass_rate = (df['결과'] == 'O').mean() * 100
    print(df.to_string(index=False))
    print(f'\n\n 통과율: {pass_rate:.0f}% ({int(pass_rate/100*len(test_cases))}/{len(test_cases)})')
    return df

eval_bot = FAQChatbot()
eval_df = evaluate_chatbot(eval_bot, TEST_CASES)
```

대학 리스트의 초기화 완료

질문 결과

수강신청 최대 학점은? [21학점, 24학점] 수강신청 최대 학점은 일반적으로 21학점까지 가능합니다. \n\n다만, 학년 학기 성적이 우수...
성적 이의신청 기간은? [1주일] 성적 이의신청은 성적 발표 후 1주일 이내에 가능합니다. \n\n학생정보실에서 온라인으로 이의신청...
장학금 성적 기준 알려줘 [3.8, 3.5] 성적 장학금 기준은 다음과 같습니다. \n\nA+ 성적 장학금: 학년 학기 12학점 이상 수강...
오늘 점심 메뉴 추천해줘 [학생자, 문의] 죄송합니다. 점심 메뉴 추천은 학생자 업무 범위에 해당하지 않습니다. \n\n메뉴 내용은 학생자...

통과율: 100% (4/4)

Simple 3 Python 3 (pykernel) Idle Initialized (additional servers needed) Instance MEM 67%

Cookie Preferences Mode: Command Ln 17, Col 48 bedrock-p1-lab3-student.ipynb

▼ 제공 코드

```
# [✓] [제공 코드] ipywidgets 채팅 UI - FAQChatbot과 직접 대화해보세요
import html as _html
import ipywidgets as widgets
from IPython.display import display

ui_bot = FAQChatbot()          # 위에서 완성한 챗봇 클래스 사용
_bubbles = []                  # 말풍선 HTML 누적

chat_log = widgets.HTML()
text_in = widgets.Text(placeholder='질문을 입력하고 Enter 또는 [전송]을 누르세요',
                        layout=widgets.Layout(width='65%'))
send_btn = widgets.Button(description='전송', button_style='primary')
reset_btn = widgets.Button(description='새 대화')

def _render():
    chat_log.value = (
        '<div style="height:340px; overflow-y:auto; border:1px solid #ddd; '
        'border-radius:8px; padding:12px; background:#fafafa; font-family:sans-serif;">'
        + ''.join(_bubbles) + '</div>')

def _bubble(role, text, faqs=None):
    text = _html.escape(text).replace('\n', '<br>')
    if role == 'user':      # 오른쪽 파란 말풍선
        return (f'<div style="text-align:right; margin:6px 0;"><span style="display:inline-block; '
                f'background:#3B82F6; color:#fff; padding:8px 12px; border-radius:14px 14px 2px 14px; '
                f'max-width:75%; text-align:left;">{text}</span></div>')
    src = ''
    if faqs:                # 근거 FAQ 출처 표시
        src = (f'<div style="font-size:11px; color:#888; margin-top:4px;">'
                f'🔗 참조 FAQ: [{_html.escape(faqs[0]["category"])}] {_html.escape(faqs[0]["question"])}</div>')
    return (f'<div style="text-align:left; margin:6px 0;"><span style="display:inline-block; '
            f'background:#e9e9ef; color:#222; padding:8px 12px; border-radius:14px 14px 14px 2px; '
            f'max-width:75%;">🤖 {text}{src}</span></div>')

def _on_send(_):
    q = text_in.value.strip()
    if not q:
```

```

        return
    text_in.value = ''
    _bubbles.append(_bubble('user', q))
    _bubbles.append('<div style="color:#aaa; font-size:12px;">답변 생성
중...</div>')
    _render()
    answer, faqs = ui_bot.chat(q)          # ← 여기서 Bedrock 호출
    _bubbles.pop()                        # "생성 중..." 제거
    _bubbles.append(_bubble('bot', answer, faqs))
    _render()

def _on_reset(_):
    ui_bot.reset()
    _bubbles.clear()
    _bubbles.append('<div style="color:#888;">새 대화를 시작합니다. 무엇이든
물어보세요!</div>')
    _render()

send_btn.on_click(_on_send)
reset_btn.on_click(_on_reset)
import warnings
with warnings.catch_warnings():          # on_submit의 DeprecationWarn
ing 숨김 (동작에는 문제 없음)
    warnings.simplefilter('ignore', DeprecationWarning)
    text_in.on_submit(_on_send)          # Enter 키로 전송
_on_reset(None)
display(widgets.VBox([chat_log, widgets.HBox([text_in, send_btn, reset_b
tn]))])

```

